

Muhammad Houzair Koussa

Software engineer

Founder @ thephilosophicalcode.com

houzair.me

The Curry-Howard Correspondence

About

Overview of a deep connection between computation and logic

Attributed to Haskell Curry and William Howard

Common slogans:

Types as propositions

Programs as proofs

Agenda

Logic:

- Arguments and formalisation

- Propositions and connectives

- Proofs

The correspondence:

- Types as propositions

- Programs as proofs

Miscellaneous

Heads up

Rudimentary overview. I'll be skipping over lots of subtleties. Ask questions!

There are varying views on logic and proofs within maths, computer science, philosophy and linguistics. We'll be sticking to classical views

I'm assuming basic knowledge of types and programs (TypeScript as language of illustration)

Logic: arguments and formalisation

Either Sun is a frontend engineer or Sun is a backend engineer

If Sun is a frontend engineer, then Sun loves VueJS

If Sun is a backend engineer, then Sun loves PostgreSQL

So, either Sun loves VueJS or Sun loves PostgreSQL

Either Dan comes to this meet-up or Dan goes shopping

If Dan comes to this meet-up, then Dan might have lunch with us

If Dan goes shopping, then Dan's wallet won't be happy

So, either Dan might have lunch with us or Dan's wallet won't be happy

Logic: arguments and formalisation

Either A or B

If A, then C

If B, then D

So, either C or D

Formalisation strips away non-logical details to leave an argument's *logical form*

We logically reason via that form. Content does not matter

We can freely swap the contents while the argument remains valid

Truth (soundness) is a different story

Logic: propositions and connectives

Propositions are things like 'A', 'B', 'If A, then C' or 'A or B'

Connectives are the logical operators left after formalisation

In our case, these are 'and', 'or' and 'if...then'

We carry out logical proofs using only these propositions and connectives

Logic: proofs - intuitions

Not all proofs are equal. Some proofs are more equal than others

Case 1: a solar company “scientifically proving” that their panels are the best

Case 2: a step-by-step mathematical proof on why 2 is a prime number

Intuitively, we trust 2 more than 1

“It’s actually difficult to provide proof first hand. Unless it’s maths, something you can *demonstrate on paper*”

Logic: proofs

A few characteristics of logical proofs:

Prove a proposition (conclusion) from a set of propositions (premises)

Step-by-step: mechanical deduction using one proof rule at a time

Rigid and rigorous - only proofs abiding by the rules are allowed

Truth preserving - we never go from a state of truth to falsehood

There are two proof rules for each connective - introduction and elimination

Introduction rule - how and when can we *add* a connective

Elimination rule - how and when can we *remove* a connective

Logic: proofs - and

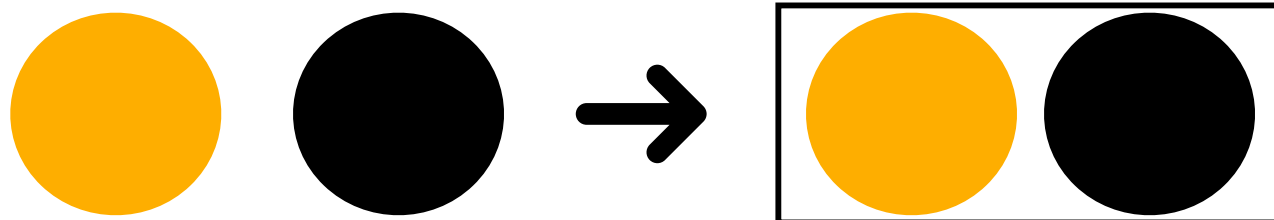
Introduction

$A, B \rightarrow A \text{ and } B$

If we know that “Bruno is a software engineer”

If we also know that “Bruno is Mauritian”

We know that “Bruno is a software engineer and Bruno is Mauritian”



Logic: proofs - and

Elimination

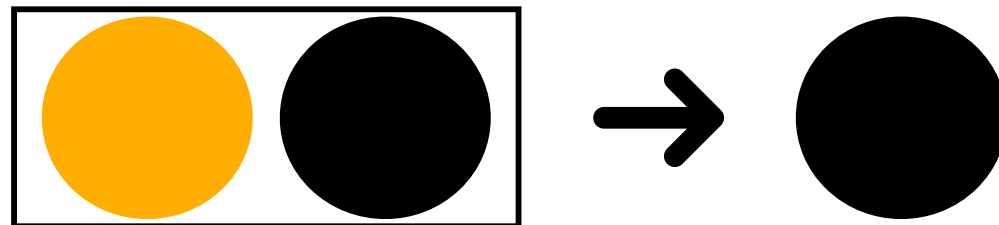
$A \text{ and } B \rightarrow A$

$A \text{ and } B \rightarrow B$

If we know that “Bruno is a software engineer and Bruno is Mauritian”

We know that “Bruno is a software engineer”

We also know that “Bruno is Mauritian”



Logic: proofs - or

Introduction

$$A \rightarrow A \text{ or } B$$

Only one branch of “A or B” needs to be true
for the whole of “A or B” to be true

If we know that “Bun is fast”

We know that “Bun is fast or Node is slow”

A	B	A or B
F	F	F
T	F	T
F	T	T
T	T	T

Logic: proofs - or

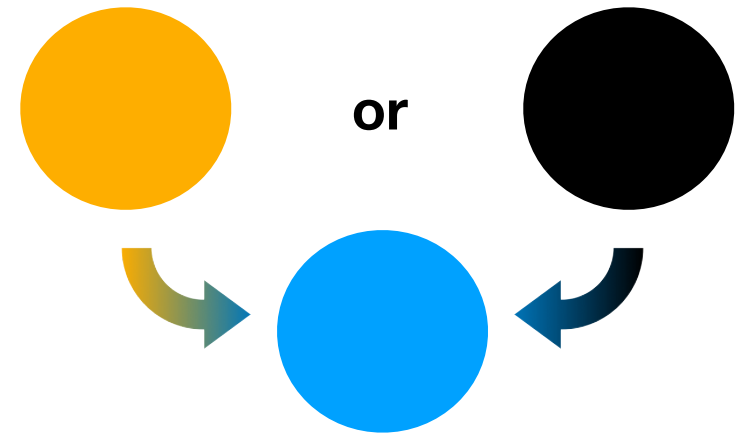
Elimination

$A \text{ or } B, \text{ if } A \text{ then } C, \text{ if } B \text{ then } C \rightarrow C$

“A or B” has two branches

If in both branches, we can deduce C, then

we can deduce “C” from “A or B”



If we know that “Bun is fast or Node is slow” and both “Bun is fast” and “Node is slow” imply “JS is cool”. We know that “JS is cool”

Logic: proofs - if...then

Introduction

Assuming A then B $\rightarrow$ if A then B

If we assume A and somehow deduce B (from the assumption), We can deduce that A implies B (if A then B)

If we know that the assumption that “Ced uses PostgreSQL” leads to the conclusion that “Ced knows SQL”

We know that “if Ced uses PostgreSQL, then Ced knows SQL”

Logic: proofs - if...then

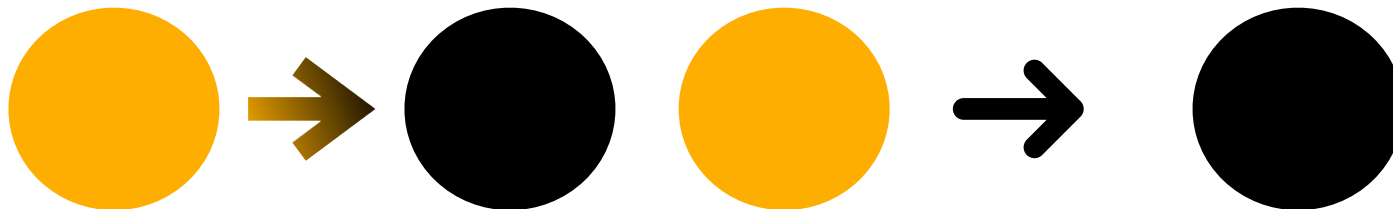
Elimination

A, if A then B $\rightarrow$ B

If we know that “If Alex is in Port Louis, then Alex is in Mauritius”

And if we also know that “Alex is in Port Louis”

We know that “Alex is in Mauritius”



Logic: proofs - example 1

Either A or B

If A, then C

If B, then D

So, either C or D

1. A or B, if A then C, if B then D (premises)

2. Assume A

2.1 C (if...then elimination from 1 and 2)

2.2 C or D (or introduction from 2.1)

3. Assume B

3.1 D (if...then elimination from 1 and 3)

3.2 C or D (or introduction from 3.1)

4. C or D (or elimination from 1, 2.2 and 3.2)

Logic: proofs - example 2

A and B

If A, then C

If B, then D

So, (C and D) or E

1. A and B, if A then C, if B then D (premises)

2. A (and elimination from 1)

3. B (and elimination from 1)

4. C (if...then elimination from 1 and 2)

5. D (if...then elimination from 1 and 3)

6. C and D (and introduction from 4 and 5)

7. (C and D) or E (or introduction from 6)

The correspondence: types as propositions

Logic	Computation
Proposition A	Generic type A
A and B	Tuple type [A, B]
A or B	Tagged union type {t: "L": v: A} {t: "R": v: B}
If A then B	Function type (a: A) => B

The correspondence: types as propositions

A and B - why not A & B instead of [A, B]?

Logical “A and B” is characterised as set-theoretic intersection “ $A \cap B$ ”

TypeScript’s “A & B” is not set-theoretic. It’s structural

“ $A \cap B$ ” only contains the *common* properties between set A and set B

“A & B” contains *all* properties between type A and type B

$\{\text{foo}: 1\} \& \{\text{bar}: 2\} \neq \{\}$

$\{\text{foo}: 1\} \& \{\text{bar}: 2\} = \{\text{foo}: 1, \text{bar}: 2\}$

The correspondence: types as propositions

A or B - why not A & B instead of $\{t: \text{"L"}: v: A\} \mid \{t: \text{"R"}: v: B\}$?

Remember the proof rule for or-elimination. Given “A or B”, we can prove C only if we derive C from both branches A and B

Intuitively, we should be able to *conditionally distinguish* between the two branches and derive C from each

AFAIK, we don't really have an “is A” or “is B” to conditionally distinguish between the two generic types from “A | B”

With the tagged union, we can do so with the “t” tag

The correspondence: programs as proof

1. A or B, if A then C, if B then D

2. Assume A

2.1 C

2.2 C or D

3. Assume B

3.1 D

3.2 C or D

4. C or D

```
type Or<A, B> = { t: "L"; v: A } | { t: "R"; v: B };
const L = <A, B>(a: A): Or<A, B> => ({ t: "L", v: a });
const R = <A, B>(b: B): Or<A, B> => ({ t: "R", v: b });

const proof = <A, B, C, D>(
  aOrB: Or<A, B>, // 1. Premises
  ifAThenC: (a: A) => C,
  ifBThenD: (b: B) => D,
): Or<C, D> => {
  let cOrD: Or<C, D>;
  switch (aOrB.t) {
    case "L": { // 2. Assume A
      const a = aOrB.v;
      const c: C = ifAThenC(a); // 2.1 C
      cOrD = L<C, D>(c); // 2.2 C or D
      break;
    }
    case "R": { // 3. Assume B
      const b = aOrB.v;
      const d: D = ifBThenD(b); // 3.1 D
      cOrD = R<C, D>(d); // 3.2 C or D
      break;
    }
  }
  return cOrD; // 4. C or D
};
```

The correspondence: programs as proof

1. A and B, if A then C, if B then D

2. A

3. B

4. C

5. D

6. C and D

7. (C and D) or E

```
type Or<A, B> = { t: "L"; v: A } | { t: "R"; v: B };
const L = <A, B>(a: A): Or<A, B> => ({ t: "L", v: a });

const proof = <A, B, C, D, E>(  
  aAndB: [A, B], // 2, 3.           // 1. Premises  
  ifAThenC: (a: A) => C,  
  ifBThenD: (b: B) => D,  
): Or<[C, D], E> => {  
  const [a, b] = aAndB;           // 2. A, 3. B  
  const c: C = ifAThenC(a);       // 4. C  
  const d: D = ifBThenD(b);       // 5. D  
  const cAndD: [C, D] = [c, d];   // 6. C and D  
  let cAndDOrE: Or<[C, D], E>;  
  cAndDOrE = L<[C, D], E>(cAndD); // 7. (C and D) or E  
  return cAndDOrE;  
};
```

Miscellaneous: ideally vs actually

Ideally, every time we type check a program, we're running a logical proof

Actually, most languages and type systems are not that “pure” and their implementations vary widely

For example, TypeScript's “any” type. Everything goes. It's not really a proof

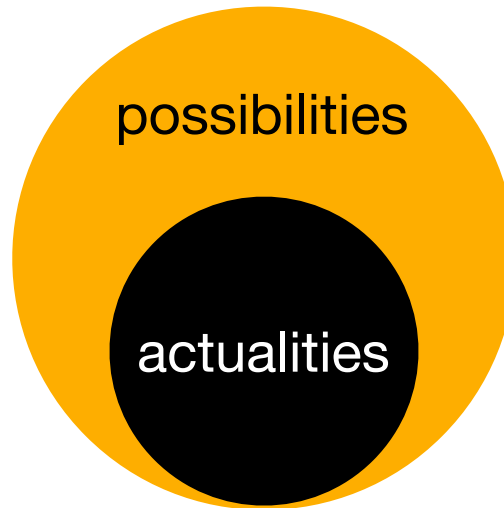
Different type systems correspond to different logics - some may be inconsistent or even trivial.

Miscellaneous: domain modelling

We usually use typed interfaces to model domains

Since we may view types as propositions used in proofs, we can also view these interfaces as modelling what's provable in our domain

So, interfaces describe the domain's actualities and its (provable) possibilities



Miscellaneous: domain modelling

An interface can “prove” too much giving rise to unwanted possibilities

```
const user: {  
  name: string;  
  age: number;  
  email: string;  
} = {  
  name: "",  
  age: -1,  
  email: "hello world!"  
}
```

Tightening what’s “provable” is akin to reducing unwanted possibilities

```
{  
  name: z.string().min(1),  
  age: z.number().int().nonnegative().max(150),  
  email: z.email()  
}
```

Miscellaneous: automated theorem proving

Ideally, every logical proof has a program

There are languages that try to abide by this ideal: LEAN, Agda, Coq

We can represent complex logical and mathematical proofs in these languages and run them to check for validity

That's it, folks!

Slides + notes @ houzair.me/talks